

UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

# Lab 9: Midterm 2 Review

Wednesday, November 4, 2026

# Reminders

- **Midterm 2 is tonight, 7-9pm!**
  - **Mahar 108**
  - Advice:
    - Write legibly (reduces risk of misgrading)
    - Write the types of function parameters and return values
    - The order of the questions doesn't mean anything, feel free to do them in whatever order is best for you
- **Homework 8 questionnaires are due Monday at 11:59pm!**
  - CATME
  - Google Form

# Topic Poll

Which topics do you most want to review?

- Property-Based Testing
- Iterators
- Generators
- Observer, Singleton, and Model-View-Controller Design Patterns

# Property-Based Testing

# Exercise: Property-Based Testing

Write property-based tests for the function

```
partition(arr: number[], p: number)
```

with the following specification:

- The array elements and **`p`** are assumed to be integers between **`0`** and **`arr.length-1`**.
- The function rearranges the array in place, so that a number with value > **`p`** will not appear before a number with value <= **`p`**, and a number with value < **`p`** will not appear after a number with value >= **`p`**.

Try to write a complete set of tests, adding comments to clearly indicate the purpose of each test.

# Solution: Property-Based Testing

Tests:

1. Check that the original and resulting array have the same length.
2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling **`partition`** and comparing.
3. Check that the result has elements less than **p**, then elements equal to **p**, then elements greater than **p** (any of those may be missing).

# Solution: Property-Based Testing

1. Check that the original and resulting array have the same length.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  // ...
}
```

We will write a helper function **checkSame** for the first two tests we have described. We begin by simply checking if the two arrays have the same length.

# Solution: Property-Based Testing

2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling partition and comparing.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  const counts = new Array<number>(a.length).fill(0);
  // ...
}
```

Helper function **`isValid`** checks if **`e`** is an integer between **`0`** and **`a.length-1`**.
**`counts`** will track the number of times each number appears, initialized to **`0`**s.

# Solution: Property-Based Testing

2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling partition and comparing.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  const counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  // ...
}
```

We use **`forEach`** to check that each element in **`b`** (result from partition) is valid and to update **`counts`**, tracking the number of times each value appears in **`b`**.

# Solution: Property-Based Testing

2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling partition and comparing.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  const counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  a.forEach(e => assert(--counts[e] >= 0));
}
```

We call **forEach** on **a** (the original array) and ensures that each element does not appear more often in **a** than in **b**.

# Solution: Property-Based Testing

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  const counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  a.forEach(e => assert(--counts[e] >= 0));
}
```

Is this sufficient to check that elements have the same frequencies in both arrays?

# Solution: Property-Based Testing

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  const counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  a.forEach(e => assert(--counts[e] >= 0));
}
```

Is this sufficient to check that elements have the same frequencies in both arrays?

Yes. The last check ensures that no element appears more often in **a** than it does in **b**. If there was such an element, **counts** would have at least one non-zero count.

# Solution: Property-Based Testing

3. Check that the result has elements less than **p**, then elements equal to **p**, then elements greater than **p** (any of those may be missing).

```
function checkPivot(a: number[], p: number): void {
  let i = a.length;
  if (i === 0) return;
  while (i > 0 && a[--i] > p) {}    //all > p
  while (i >= 0 && a[i] === p) --i; //all = p
  while (i >= 0) { assert(a[i] < p); --i; }
}
```

For the third test, we can iterate through the resulting array from right to left. We use while loops to first iterate through all elements greater than **p**, then all elements equal to **p**, and lastly assert that all remaining elements are less than **p**.

# Solution: Property-Based Testing

Tests:

1. Check that the original and resulting array have the same length
2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before partition and comparing:
3. Check that the result has elements less than **p**, then elements equal to **p**, then elements greater than **p** (any of those may be missing).

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) =>
    0 <= e && e < a.length && e%1 === 0;
  let f = new Array<number>(a.length).fill(0);
  b.forEach(e => { assert(inRange(e)); ++f[e]; });
  a.forEach(e => assert(--f[e] >= 0));
}

function checkPivot(a: number[], p: number): void {
  let i = a.length; if (i === 0) return;
  while (i > 0 && a[--i] > p) {}    // all > p
  while (i >= 0 && a[i] === p) --i; // all = p
  while (i >= 0) { assert(a[i] < p); --i; }
}
```

You would then use these two functions to test the output of any given partition function to check if it follows the specifications.

# Iterators

# Exercise: Iterators



Write a function `makeChain` that takes in an array of iterables and returns an instance of a custom `ChainIterator` class.

The `ChainIterator` class should be an iterable iterator that yields items from the first iterable until it runs out. It should then yield items from the next iterable in the given array, repeating this process until the end of the array.

You may assume the given array is not empty.

```
class ChainIterator<T> {
  // TODO: Implement this class
}

function makeChain<T>(iterables: Iterable<T>[]): ChainIterator<T> {
  // TODO: Implement this function
  return new ChainIterator(); // replace as needed
}
```

# Solution: Iterators



```
class ChainIterator<T> implements IterableIterator<T> {
  private _idx: number;
  private _curr_iterator: Iterator<T>;
  private _iterables: Iterable<T>[];

  // ...
```

We first specify `ChainIterator` implements the `IterableIterator` interface.

We then define 3 private variables, tracking the index of the current iterable (`_idx`), the iterator of the current iterable (`_curr_iterator`), and the array of iterables (`_iterables`).

# Solution: Iterators



```
class ChainIterator<T> implements IterableIterator<T> {
  private _idx: number;
  private _curr_iterator: Iterator<T>;
  private _iterables: Iterable<T>[];

  constructor(iterables: Iterable<T>[]) {
    this._idx = 0;
    this._curr_iterator = iterables[0][Symbol.iterator]();
    this._iterables = iterables.map(x => x); // .map() is optional
  }

  // ...
```

We initialize `_idx` to `0` and get the iterator for the first iterable with `[Symbol.iterator]()`.

We then store the array of iterables as well as a private property.

You may want to copy the given array (i.e using `.map`, or `[...iterables]`) to prevent outside changes to the array from affecting our iterator.

# Solution: Iterators



```
class ChainIterator<T> implements IterableIterator<T> {
  private _idx: number;
  private _curr_iterator: Iterator<T>;
  private _iterables: Iterable<T>[];

  constructor(iterables: Iterable<T>[]) {
    this._idx = 0;
    this._curr_iterator = iterables[0][Symbol.iterator]();
    this._iterables = iterables.map(x => x); // .map() is optional
  }

  [Symbol.iterator]() { return this; }

  // ...
```

We implement `[Symbol.iterator]()` so that our class implements the `Iterable` interface, returning `this` since our class is itself an iterator.

# Solution: Iterators



```
class ChainIterator<T> implements IterableIterator<T> {
  private _idx: number;
  private _curr_iterator: Iterator<T>;
  private _iterables: Iterable<T>[];
  // ...

  next(): IteratorResult<T> {
    let iter_result = this._curr_iterator.next();
    while (iter_result.done) {

      // ...
```

For `next()`, we access the next item from the current iterator, which is an `IteratorResult`.

If the `IteratorResult` indicates that the current iterator is exhausted, we want to find the next iterable that is not done.

# Solution: Iterators



```
class ChainIterator<T> implements IterableIterator<T> {
  private _idx: number;
  private _curr_iterator: Iterator<T>;
  private _iterables: Iterable<T>[];
  // ...

  next(): IteratorResult<T> {
    let iter_result = this._curr_iterator.next();
    while (iter_result.done) {
      this._idx++;
      if (this._idx >= this._iterables.length) {
        return { done: true, value: undefined };
      }

      // ...
```

We increment `_idx` to move to the next iterable.

If `_idx` has reached the end of the iterables array, we will return the `IteratorResult` indicating our iterator has completed.

# Solution: Iterators



```
class ChainIterator<T> implements IterableIterator<T> {
  private _idx: number;
  private _curr_iterator: Iterator<T>;
  private _iterables: Iterable<T>[];
  // ...

  next(): IteratorResult<T> {
    let iter_result = this._curr_iterator.next();
    while (iter_result.done) {
      this._idx++;
      if (this._idx >= this._iterables.length) {
        return { done: true, value: undefined };
      }
      this._curr_iterator = this._iterables[this._idx][Symbol.iterator]();
      iter_result = this._curr_iterator.next();
    }
    // ...
```

Otherwise, we obtain the iterable at index `_idx` and retrieve its iterator, storing it at the new `_curr_iterator`. We then retrieve the next item from this iterator.

# Solution: Iterators



```
class ChainIterator<T> implements IterableIterator<T> {
  private _idx: number;
  private _curr_iterator: Iterator<T>;
  private _iterables: Iterable<T>[];
  // ...

  next(): IteratorResult<T> {
    let iter_result = this._curr_iterator.next();
    while (iter_result.done) {
      this._idx++;
      if (this._idx >= this._iterables.length) {
        return { done: true, value: undefined };
      }
      this._curr_iterator = this._iterables[this._idx][Symbol.iterator]();
      iter_result = this._curr_iterator.next();
    }
    return { done: false, value: iter_result.value };
  }
}
```

Lastly, we return the value from `iter_result` as the value of an `IteratorResult`.

# Solution: Iterators

```
function makeChain<T>(iterables: Iterable<T>[]): ChainIterator<T> {
  return new ChainIterator(iterables);
}
```

We can now implement the function `makeChain` by making a new instance of the `ChainIterator` class we have implemented.

# Generators

# Exercise: Generators

Write a generator function `chainGenerator` for the `ChainIterator` problem. The returned Generator should behave the same way as `ChainIterator` from the "Iterators" exercise.

- Takes in an array of `Iterable`s.
- Yield items from the first iterable until it runs out, then the second, etc.
- Repeat for each array in `iterables`.
- Assume `iterables` is not empty.

```
function* chainGenerator<T>(iterables: Iterable<T>[]): Generator<T> {
  // TODO: Implement this function
}
```

# Solution: Generators

```
function* chainGenerator<T>(iterables: Iterable<T>[]): Generator<T> {
  for (const iterable of iterables) {

    // ...
  }
}
```

We will first iterate through the array of `iterables` provided.

Our goal is to move from the start of the array to the end, yielding all items from the current iterable before moving on to the next iterable.

# Solution: Generators

```
function* chainGenerator<T>(iterables: Iterable<T>[]): Generator<T> {
  for (const iterable of iterables) {
    const currIterator = iterable[Symbol.iterator]();
    let currVal = currIterator.next();

    // ...
  }
}
```

For the current iterable, we will use `[Symbol.iterator]()` to retrieve its iterator. We then store the first `IteratorResult` as the variable `currVal`.

# Solution: Generators

```
function* chainGenerator<T>(iterables: Iterable<T>[]): Generator<T> {
  for (const iterable of iterables) {
    const currIterator = iterable[Symbol.iterator]();
    let currVal = currIterator.next();
    while (!currVal.done) {
      yield currVal.value;
      currVal = currIterator.next();
    }
  }
}
```

While the current iterator has items remaining, we yield its latest value. When execution resumes, we advance the iterator and update `currVal` to the next value.

Once the outer `for` loop terminates, all of the iterables will be exhausted. The generator will then automatically yield `{done: true, value: undefined}` to indicate that it has ran out of values.

# Solution: Generators

```
function* chainGenerator<T>(iterables: Iterable<T>[]): Generator<T> {
  for (const iterable of iterables) {
    for (const item of iterable) {
      yield item;
    }
  }
}
```

What about this implementation? Does this also work?

Yes! Since we have an array of iterables, we can loop through each iterable, yielding its items. This demonstrates how generators can sometimes be easier to work with iterators.

# Solution: Generators

In fact, the following also does the same thing.

```
function* chainGenerator<T>(iterables: Iterable<T>[]): Generator<T> {
  for (const iterable of iterables) {
    yield* iterable;
}
```

`yield*` successively yields values from the given iterable until it is exhausted.

# Solution: Generators

```
next(): IteratorResult<T> {
  let iter_result = this._curr_iterator.next();
  while (iter_result.done) {
    this._idx++;
    if (this._idx >= this._iterables.length) {
      return { done: true, value: undefined };
    }
    this._curr_iterator = this._iterables[this._idx][Symbol.iterator]();
    iter_result = this._curr_iterator.next();
  }
  return { done: false, value: iter_result.value };
}
```

Comparing the generator solution to the `.next()` method of `ChainIterator`, it's clear how generators allow a simpler implementation of the same problem.

Tracking the current index in `iterables` and storing an iterator for the current iterable are not necessary when using a generator function.

# Observer, Singleton, MVC

# Question: Observer Pattern 1

Which scenario best captures the core problem the Observer pattern is designed to solve?

A. Ensuring multiple objects can modify the same data concurrently

B. Allowing one object to notify multiple dependent objects automatically when its state changes

C. Reducing the number of classes in a system

D. Enforcing strict control over object creation

# Answer: Observer Pattern 1

Which scenario best captures the core problem the Observer pattern is designed to solve?

A. Ensuring multiple objects can modify the same data concurrently

B. Allowing one object to notify multiple dependent objects automatically when its state changes

C. Reducing the number of classes in a system

D. Enforcing strict control over object creation

**Answer: B**

# Question: Observer Pattern 2

Which statement best explains the interaction flow between Subject and Observers?

A. The Subject directly modify the internal state of the Observers

B. The Observers pull data from the Subject when needed

C. The Subject notifies the Observers, and the Observers react by updating themselves

D. Observers continuously request updates in a loop

# Answer: Observer Pattern 2

Which statement best explains the interaction flow between Subject and Observers?

A. The Subject directly modify the internal state of the Observers

B. The Observers pull data from the Subject when needed

C. The Subject notifies the Observers, and the Observers react by updating themselves

D. Observers continuously request updates in a loop

**Answer: C**

# Question: MVC 1

How is the Model–View–Controller pattern commonly related to the Observer pattern?

A. The Controller replaces the need for any notification system

B. The Model often uses Observer-like behavior to notify Views of changes

C. The Model directly modifies the Views without communication rules

D. The View continuously polls the Model for updates

# Answer: MVC 1

How is the Model–View–Controller pattern commonly related to the Observer pattern?

A. The Controller replaces the need for any notification system

B. The Model often uses Observer-like behavior to notify Views of changes

C. The Model directly modifies the Views without communication rules

D. The View continuously polls the Model for updates

**Answer: B**

# Question: MVC 2

Which part of the pattern is responsible for subscribing and unsubscribing Views to/from the Model?

A. Model

B. View

C. Controller

# Question: MVC 2

Which part of the pattern is responsible for subscribing and unsubscribing Views to/from the Model?

A. Model

B. View

C. Controller

**Answer: C**

# Question: MVC 3

Open ended: What are the individual responsibilities of the Model, the View, and the Controller in the Model-View-Controller design pattern?

# Question: MVC 3

Open ended: What are the individual responsibilities of the Model, the View, and the Controller in the Model-View-Controller design pattern?

**Answer:**

- The Model handles data and business logic
- The View displays data
- The Controller handles input and invokes the Model and View

# Question: Singleton Pattern

Open ended: What is the main purpose of the Singleton pattern?

# Answer: Singleton Pattern

Open ended: What is the main purpose of the Singleton pattern?

**Answer:** To ensure a class has exactly one instance and provide controlled, global access to it